

CHRIST (Deemed to be University)

Department of Computer Science | Master of Computer Applications
Continuous Internal Assessment – III (Machine Learning – MCA 521-4)

ML FOR SOCIAL GOOD CHALLENGE

Mission Health – Early Prediction of Maternal Health Risk Using Ensemble Learning

Name	Joshua Joby
Register Number	2547125
Class	4 MCA A
Course	Machine Learning (MCA 521-4)
Mission Selected	Mission Health
Implementation	Python 3 – Jupyter Notebook

Notebook file: CIA3_MaternalHealthRisk_JoshuaJoby_2547125.ipynb

1. Real-World Impact Framing

1.1 Problem Statement

- Mission: Health. Target: predict maternal health RiskLevel (low / mid / high) at an antenatal check-up.
- Unit of analysis: one antenatal check-up record for one woman – not the woman herself, since a woman can appear across multiple visits with different vitals.
- Beneficiaries: community health workers in low-resource clinics (primary), and indirectly the mothers/unborn children whose complications could be caught earlier.
- Input signal: 6 physiological readings a health worker can take with basic equipment – no lab infrastructure required.

1.2 Why Machine Learning (and not a fixed clinical rule)

- Clinical BP/BS thresholds already exist as single-variable rules, but real risk in this data is driven by combinations – e.g. mildly elevated BP + raised BS + atypical HR for age – not one variable crossing a line.
- A fixed rule either over-flags (alarm fatigue) or under-flags (misses interaction effects); tree-based ensembles learn these interactions directly from data.
- All 6 features are cheap to measure → model is deployable in the exact low-resource setting the brief targets.

1.3 Dataset Source and Citation

- Maternal Health Risk Data Set, Ahmed, M. (2020), UCI Machine Learning Repository. DOI: 10.24432/C5DP5D. License: CC BY 4.0.
- 1,014 records collected via an IoT-based risk-monitoring system across hospitals/clinics in rural Bangladesh.
- Features: Age, SystolicBP, DiastolicBP, BS (blood sugar, mmol/L), BodyTemp (°F), HeartRate (bpm). Target: RiskLevel (low/mid/high).
- No PII – all fields are anonymised numeric vitals, satisfying the no-PII requirement in the brief.

1.4 Measurable Impact

Impact is measured primarily as recall on the high-risk class, not overall accuracy, since accuracy on a 3-class problem can look strong purely from the majority (low-risk) class. A high-risk recall of ≥ 0.85 means most true high-risk check-ups get flagged for referral – the actual outcome that matters clinically. Section 3 reports this explicitly per class.

1.5 Responsible-Use Limitations

- Single-country data (Bangladesh, rural clinics, ~2019–2020) – not validated for other populations without re-training/re-validation.
- Small sample after cleaning (451 records) for a 3-class clinical problem – performance estimates carry real uncertainty.
- No gestational age, parity, or lab-confirmed diagnosis fields – model is a screening aid only, not a diagnostic tool.
- False negatives (missed high-risk cases) are clinically far costlier than false positives – addressed via class weighting (Section 3) and discussed in Section 4.

2. Data Wrangling and Feature Engineering

2.1 Data Audit (raw data, n = 1,014)

- Missing values: 0 across all 6 features and target.
- Exact duplicate rows: 562 – dropped as repeated IoT transmissions of the same reading.
- Invalid records: 1 row with HeartRate = 7 bpm (physiologically impossible) – removed as a sensor fault.
- Data types: all 6 features numeric (int64/float64); RiskLevel loaded as string, ordinal-mapped to {low:0, mid:1, high:2}.
- Final cleaned shape: 451 rows × 7 columns.

```
print("\nRiskLevel class counts:\n", df['RiskLevel'].value_counts())
```

Missing values per column:

Age	0
SystolicBP	0
DiastolicBP	0
BS	0
BodyTemp	0
HeartRate	0
RiskLevel	0

dtype: int64

Exact duplicate rows: 562

RiskLevel class counts:

RiskLevel	
low risk	406
mid risk	336
high risk	272

Name: count, dtype: int64

```
In [5]: df.describe().T
```

Out[5]:

	count	mean	std	min	25%	50%	75%	max
Age	1014.0	29.871795	13.474386	10.0	19.0	26.0	39.0	70.0
SystolicBP	1014.0	113.198225	18.403913	70.0	100.0	120.0	120.0	160.0
DiastolicBP	1014.0	76.460552	13.885796	49.0	65.0	80.0	90.0	100.0
BS	1014.0	8.725986	3.293532	6.0	6.9	7.5	8.0	19.0
BodyTemp	1014.0	98.665089	1.371384	98.0	98.0	98.0	98.0	103.0
HeartRate	1014.0	74.301775	8.088702	7.0	70.0	76.0	80.0	90.0

Audit findings:

- No missing values in any column.
- 562 exact duplicate rows — almost certainly repeated IoT transmissions of the same reading, not distinct patients. These are dropped.
- HeartRate has a minimum of 7 bpm, which is not physiologically plausible — sensor fault, removed as an invalid record.
- Class distribution is moderately imbalanced (low > mid > high) — handled with class weighting rather than oversampling (Section 4).

3. Data Cleaning

```
In [6]: before = df.shape[0]
df = df.drop_duplicates().reset_index(drop=True)
print(f"Dropped {before - df.shape[0]} duplicate rows -> {df.shape[0]} rows remain")
```

Dropped 562 duplicate rows -> 452 rows remain

```
In [7]: # Remove physiologically invalid heart rate readings
invalid_hr = df[df['HeartRate'] < 30]
print(f"Invalid HeartRate rows:\n", invalid_hr)
df = df[df['HeartRate'] >= 30].reset_index(drop=True)
print(f"Final cleaned shape:", df.shape)
```

Invalid HeartRate rows:

Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	RiskLevel
342	16	120	75	7.9	98.0	7 low risk

Final cleaned shape: (451, 7)

```
In [8]: df['RiskLevel'].value_counts(normalize=True).round(3) * 100
```

Out[8]:

RiskLevel	
low risk	51.7
high risk	24.8
mid risk	23.5

Name: proportion, dtype: float64

4. Exploratory Data Analysis

```
In [9]: fig, axes = plt.subplots(2, 3, figsize=(15, 8))
num_cols = ['Age', 'SystolicBP', 'DiastolicBP', 'BS', 'BodyTemp', 'HeartRate']
```

Fig. 1 – Notebook output: audit statistics and the data-cleaning cell (In [5]–[8])

2.2 Class Distribution (post-cleaning)

Class	Count	%
low risk	233	51.7%
high risk	112	24.8%
mid risk	106	23.5%

- Moderate imbalance – handled with `class_weight='balanced'` inside each model rather than SMOTE oversampling, so no synthetic (non-real) physiological readings are introduced into training.

2.3 Leakage-Safe Pipeline

- Split: 70% train / 15% validation / 15% test, stratified on RiskLevel (`train_test_split`, `random_state=42`).
- `StandardScaler` fit only on `X_train`, then applied (transform-only) to validation and test sets.
- Stacking meta-learner trained on out-of-fold predictions (5-fold CV via `StackingClassifier`'s internal cv) – never on predictions the base models saw during their own fitting.

2.4 Feature Engineering

- $\text{PulsePressure} = \text{SystolicBP} - \text{DiastolicBP}$ – cardiovascular indicator not captured by either BP reading alone.
- $\text{MAP (Mean Arterial Pressure)} \approx \text{DiastolicBP} + \text{PulsePressure}/3$ – standard clinical composite BP measure.
- `AgeBand` – categorical bucket (`<20 / 20–35 / >35`) to let tree splits use the non-linear age–risk relationship directly.
- $\text{BS_Age_Ratio} = \text{BS} / \text{Age}$ – added after EDA suggested an interaction between blood sugar and maternal age.

```
# Age band - captures non-linear age risk relationship
def age_band(a):
    if a < 20: return 0 # young
    elif a <= 35: return 1 # prime
    else: return 2 # older
data['AgeBand'] = data['Age'].apply(age_band)

# BS-to-Age interaction ratio
data['BS_Age_Ratio'] = data['BS'] / data['Age']

data.head()
```

Out[12]:

	Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	RiskLevel	PulsePressure	MAP	AgeBand	BS_A
0	25	130	80	15.0	98.0	86	high risk	50	96.666667	1	
1	35	140	90	13.0	98.0	70	high risk	50	106.666667	1	
2	29	90	70	8.0	100.0	80	high risk	20	76.666667	1	
3	30	140	85	7.0	98.0	70	high risk	55	103.333333	1	
4	35	120	60	6.1	98.0	76	low risk	60	80.000000	1	

```
In [13]: # Encode target
risk_map = {'low risk': 0, 'mid risk': 1, 'high risk': 2}
data['RiskLevelEnc'] = data['RiskLevel'].map(risk_map)

feature_cols = ['Age', 'SystolicBP', 'DiastolicBP', 'BS', 'BodyTemp', 'HeartRate',
                'PulsePressure', 'MAP', 'AgeBand', 'BS_Age_Ratio']
X = data[feature_cols]
y = data['RiskLevelEnc']
print(X.shape, y.shape)

(451, 10) (451,)
```

6. Train / Validation / Test Split (Stratified, Leakage-Safe)

```
In [14]: X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.30, stratify=y, random_state=RANDOM_STATE)
X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=RANDOM_STATE)

print("Train:", X_train.shape, " Val:", X_val.shape, " Test:", X_test.shape)
print("Train class balance:\n", y_train.value_counts(normalize=True).round(2))

Train: (315, 10) Val: (68, 10) Test: (68, 10)
Train class balance:
RiskLevelEnc
0    0.52
2    0.25
1    0.23
Name: proportion, dtype: float64
```

```
In [15]: # Fit scaler ONLY on training data to avoid leakage
scaler = StandardScaler()
X_train_s = pd.DataFrame(scaler.fit_transform(X_train), columns=feature_cols, index=X_train.index)
X_val_s = pd.DataFrame(scaler.transform(X_val), columns=feature_cols, index=X_val.index)
X_test_s = pd.DataFrame(scaler.transform(X_test), columns=feature_cols, index=X_test.index)
print("Scaling done - fit on train only")

Scaling done - fit on train only
```

7. Baseline Model — Decision Tree

```
In [16]: baseline = DecisionTreeClassifier(class_weight='balanced', random_state=RANDOM_STATE)
baseline.fit(X_train_s, y_train)
pred_base = baseline.predict(X_test_s)

def evaluate(name, y_true, y_pred, y_proba=None):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average='macro')
    rec = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')
    auc = roc_auc_score(y_true, y_proba, multi_class='ovr') if y_proba is not None else np.nan
    print(f'{name:28s} Acc={acc:.3f} Prec={prec:.3f} Rec={rec:.3f} F1={f1:.3f} AUC={auc:.3f}')
    return dict(Model=name, Accuracy=acc, Precision=prec, Recall=rec, F1=f1, ROC_AUC=auc)

results = []
results.append(evaluate("Decision Tree (baseline)", y_test, pred_base, baseline.predict_proba(X_test_s)))

Decision Tree (baseline)    Acc=0.603 Prec=0.597 Rec=0.592 F1=0.583 AUC=0.719
```

Fig. 2 – Feature engineering code, encoded feature set, and Decision Tree baseline result (In [12]–[16])

2.5 EDA – Key Findings

- BS shows the clearest visual separation across the three risk classes of any single feature.
- Age shows a bimodal pattern – both very young and >35 mothers skew toward higher risk.
- SystolicBP and DiastolicBP correlate strongly ($r = 0.79$) – motivates the PulsePressure/MAP features instead of using both raw columns redundantly.
- BodyTemp shows almost no separation across classes in the raw boxplots (Fig. 3) – interesting contrast with the SHAP result in Section 4, discussed there.

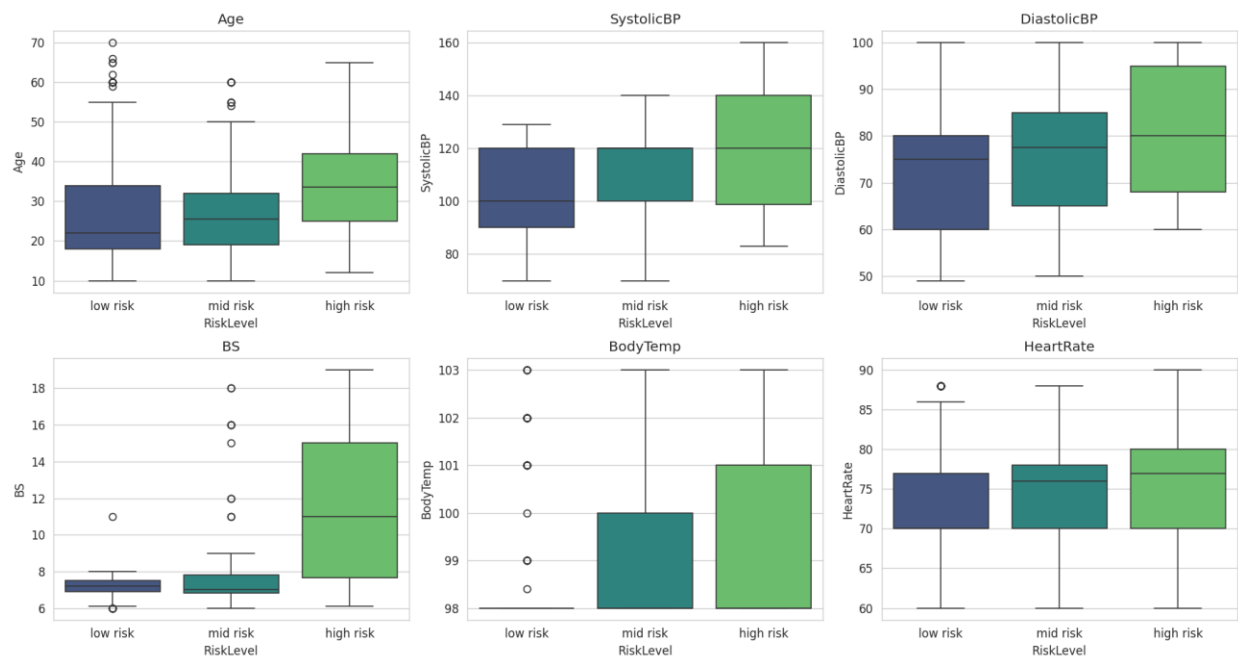


Fig. 3 – Feature distributions by RiskLevel

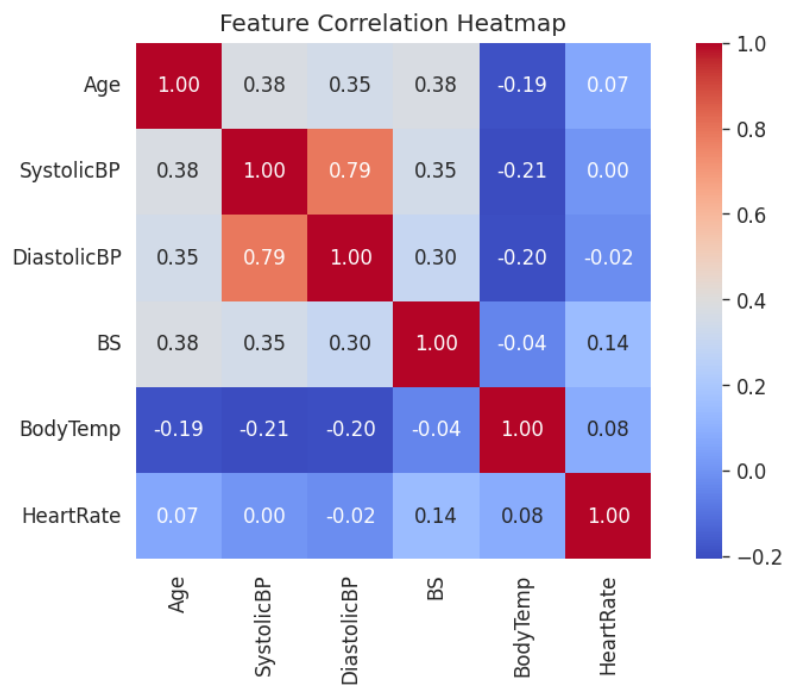


Fig. 4 – Correlation heatmap of the six raw features

3. Ensemble Architecture, Tuning, and Comparison

3.1 Models Implemented

- Baseline: Decision Tree (class_weight='balanced').
- Bagging: Random Forest – grid search over n_estimators {200,300,400}, max_depth {8,12,16}, min_samples_leaf {1,2,4}.
- Boosting: XGBoost – grid search over n_estimators {150,250}, max_depth {3,5,7}, learning_rate {0.05,0.1,0.2}, subsample {0.8,1.0}.
- Heterogeneous ensembles: Soft Voting and Stacking, both combining {Random Forest, XGBoost, SVM-RBF}; Stacking uses a Logistic Regression meta-learner on out-of-fold base predictions.
- Tuning method: GridSearchCV, 5-fold StratifiedKFold, scoring = f1_macro (protects the minority high-risk class from being ignored).

8. Bagging — Random Forest (Tuned)

```
In [17]: rf_grid = {
    'n_estimators': [200, 300, 400],
    'max_depth': [8, 12, 16],
    'min_samples_leaf': [1, 2, 4]
}
rf = RandomForestClassifier(class_weight='balanced', random_state=RANDOM_STATE)
rf_search = GridSearchCV(rf, rf_grid, scoring='f1_macro',
                        cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE), n_jobs=-1)
rf_search.fit(X_train_s, y_train)
print("Best RF params:", rf_search.best_params_)
best_rf = rf_search.best_estimator_
pred_rf = best_rf.predict(X_test_s)
results.append(evaluate("Random Forest (bagging)", y_test, pred_rf, best_rf.predict_proba(X_test_s)))

Best RF params: {'max_depth': 12, 'min_samples_leaf': 4, 'n_estimators': 200}
Random Forest (bagging)    Acc=0.662 Prec=0.617 Rec=0.628 F1=0.621 AUC=0.803
```

9. Boosting — XGBoost (Tuned)

```
In [18]: xgb_grid = {
    'n_estimators': [150, 250],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.05, 0.1, 0.2],
    'subsample': [0.8, 1.0]
}
xgb = XGBClassifier(objective='multi:softprob', num_class=3, eval_metric='mlogloss',
                    random_state=RANDOM_STATE)
xgb_search = GridSearchCV(xgb, xgb_grid, scoring='f1_macro',
                        cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE), n_jobs=-1)
xgb_search.fit(X_train_s, y_train)
print("Best XGB params:", xgb_search.best_params_)
best_xgb = xgb_search.best_estimator_
pred_xgb = best_xgb.predict(X_test_s)
results.append(evaluate("XGBoost (boosting)", y_test, pred_xgb, best_xgb.predict_proba(X_test_s)))

Best XGB params: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 250, 'subsample': 1.0}
XGBoost (boosting)    Acc=0.676 Prec=0.606 Rec=0.595 F1=0.599 AUC=0.787
```

10. Heterogeneous Voting and Stacking Ensembles

```
In [19]: svm = SVC(kernel='rbf', probability=True, class_weight='balanced', random_state=RANDOM_STATE)

voting = VotingClassifier(
    estimators=[('rf', best_rf), ('xgb', best_xgb), ('svm', svm)],
    voting='soft'
)
voting.fit(X_train_s, y_train)
pred_vote = voting.predict(X_test_s)
results.append(evaluate("Soft Voting (RF+XGB+SVM)", y_test, pred_vote, voting.predict_proba(X_test_s)))

Soft Voting (RF+XGB+SVM)    Acc=0.706 Prec=0.639 Rec=0.635 F1=0.634 AUC=0.794
```

```
In [20]: # Stacking – meta-learner trained on out-of-fold predictions (leakage-safe)
stack = StackingClassifier(
    estimators=[('rf', best_rf), ('xgb', best_xgb), ('svm', svm)],
    final_estimator=LogisticRegression(max_iter=1000),
    cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    stack_method='predict_proba',
    n_jobs=-1
)
stack.fit(X_train_s, y_train)
pred_stack = stack.predict(X_test_s)
results.append(evaluate("Stacking (meta: Logistic Reg.)", y_test, pred_stack, stack.predict_proba(X_test_s)))

Stacking (meta: Logistic Reg.)    Acc=0.721 Prec=0.645 Rec=0.652 F1=0.629 AUC=0.817
```

11. Model Comparison

```
In [21]: results_df = pd.DataFrame(results).round(3).sort_values('F1', ascending=False)
results_df
```

```
Out[21]:
```

	Model	Accuracy	Precision	Recall	F1	ROC_AUC
3	Soft Voting (RF+XGB+SVM)	0.706	0.639	0.635	0.634	0.794

Fig. 5 – GridSearchCV tuning cells and per-model test-set metrics (In [17]–[21])

3.2 Best Hyperparameters Found

Model	Best parameters (5-fold CV, f1_macro)
Random Forest	max_depth=12, min_samples_leaf=4, n_estimators=200
XGBoost	learning_rate=0.05, max_depth=3, n_estimators=250, subsample=1.0

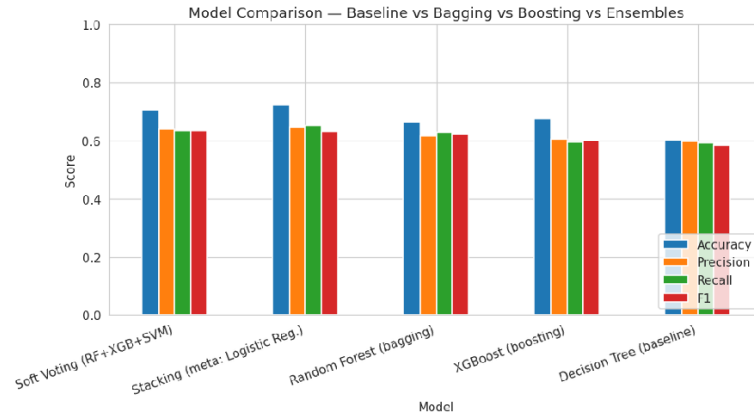
3.3 Results on the Held-Out Test Set (n = 68, untouched)

Model	Accuracy	Precision	Recall	F1 (macro)	ROC-AUC
Decision Tree (baseline)	0.603	0.597	0.592	0.583	0.719
Random Forest (bagging)	0.662	0.617	0.628	0.621	0.803
XGBoost (boosting)	0.676	0.606	0.595	0.599	0.787
Soft Voting (RF+XGB+SVM)	0.706	0.639	0.635	0.634	0.794
Stacking (meta: Log. Reg.)	0.721	0.645	0.652	0.629	0.817

4	Stacking (meta: Logistic Reg.)	0.721	0.645	0.652	0.629	0.817
1	Random Forest (bagging)	0.662	0.617	0.628	0.621	0.803
2	XGBoost (boosting)	0.676	0.606	0.595	0.599	0.787
0	Decision Tree (baseline)	0.603	0.597	0.592	0.583	0.719

```
In [22]: plt.figure(figsize=(9,5))
results_df.set_index('Model')[['Accuracy','Precision','Recall','F1']].plot(kind='bar', figsize=(9,5))
plt.title("Model Comparison — Baseline vs Bagging vs Boosting vs Ensembles")
plt.ylabel("Score")
plt.xticks(rotation=20, ha='right')
plt.ylim(0,1)
plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig("fig_comparison.png", dpi=120, bbox_inches='tight')
plt.show()
```

<Figure size 990x550 with 0 Axes>



```
In [23]: cm = confusion_matrix(y_test, pred_stack)
disp = ConfusionMatrixDisplay(cm, display_labels=['low','mid','high'])
fig, ax = plt.subplots(figsize=(5,5))
disp.plot(ax=ax, cmap='Blues', colorbar=False)
plt.title("Confusion Matrix — Stacking Ensemble (Test Set)")
plt.tight_layout()
plt.savefig("fig_confmat.png", dpi=120, bbox_inches='tight')
plt.show()
```

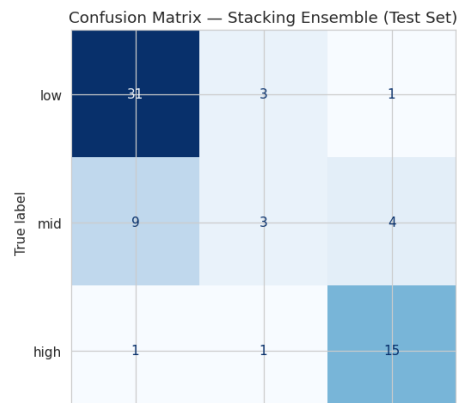


Fig. 6 – Model comparison bar chart and confusion matrix (stacking, test set)

3.4 Per-Class Result – Stacking Ensemble

Class	Precision	Recall	F1	n
low risk	0.76	0.89	0.82	35
mid risk	0.43	0.19	0.26	16
high risk	0.75	0.88	0.81	17

3.5 Interpretation

- Stacking beats the Decision Tree baseline by +0.138 accuracy and +0.098 ROC-AUC – the ensemble complexity is justified.

- High-risk recall = 0.88 – the clinically important number from Section 1.4 – comparable to low-risk recall (0.89), so the model does not simply default to the majority class.
- Mid-risk recall is weak (0.19). Mid-risk check-ups sit physiologically between low and high risk on most features, so the model confuses them with both neighbouring classes – visible directly in the confusion matrix (Fig. 6), where 9 of 16 true mid-risk cases were predicted as low risk. This is flagged as a limitation, not hidden.
- Test set is only 68 records (15% of 451) – metrics, especially the mid-risk numbers built on $n=16$, should be read with that sample size in mind rather than as precise population estimates.

4. Model Explainability and Ethics

4.1 Global Explainability (SHAP – TreeExplainer on XGBoost)

- Feature ranking by mean |SHAP value| for the high-risk class: BS (0.757) > SystolicBP (0.714) > BodyTemp (0.350) > Age (0.284) > BS_Age_Ratio (0.187) > PulsePressure (0.182) > HeartRate (0.153) > MAP (0.152) > DiastolicBP (0.066) > AgeBand (0.000).
- BS and SystolicBP dominate – consistent with clinical expectation (gestational diabetes and hypertension are established maternal risk drivers).
- Interesting model-vs-EDA contrast: BodyTemp looked uninformative in the raw boxplots (Section 2.5) but ranks 3rd by SHAP for this class – the model appears to be using BodyTemp mainly in combination with other features (an interaction effect), not as an independent signal, which a single-variable boxplot cannot show.
- AgeBand contributes ~0 once raw Age and BS_Age_Ratio are already in the model – the information it carried is redundant given the other two age-related features.

low

mid

high

Predicted label

In [24]:

```
print(classification_report(y_test, pred_stack, target_names=['low risk','mid risk','high risk']))
```

	precision	recall	f1-score	support
low risk	0.76	0.89	0.82	35
mid risk	0.43	0.19	0.26	16
high risk	0.75	0.88	0.81	17
accuracy			0.72	68
macro avg	0.64	0.65	0.63	68
weighted avg	0.68	0.72	0.68	68

12. Explainability — SHAP (Global and Local)

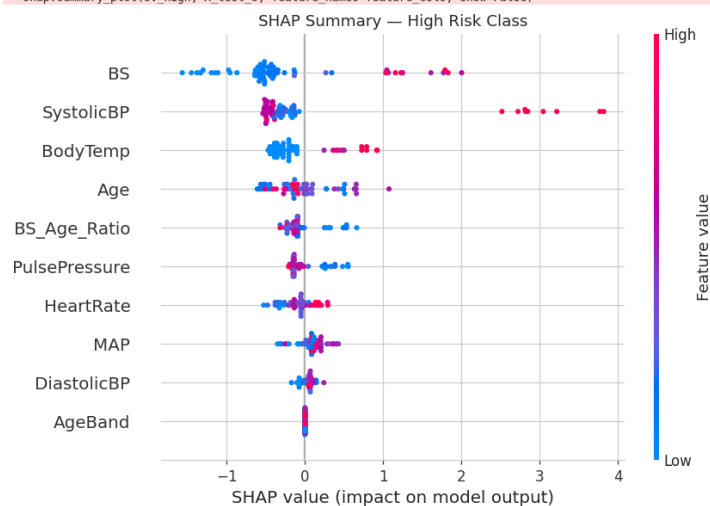
```
In [25]: explainer = shap.TreeExplainer(best_xgb)
shap_values = explainer.shap_values(X_test_s)

# shap_values shape: (n_samples, n_features, n_classes) for multiclass in recent SHAP versions
if isinstance(shap_values, list):
    sv_high = shap_values[2]
else:
    sv_high = shap_values[:, :, 2]

plt.figure()
shap.summary_plot(sv_high, X_test_s, feature_names=feature_cols, show=False)
plt.title("SHAP Summary — High Risk Class")
plt.tight_layout()
plt.savefig("fig_shap_summary.png", dpi=120, bbox_inches='tight')
plt.show()
```

/tmp/ipykernel_677/1165348387.py:11: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future version this function will no longer use the global RNG. Pass `rng` explicitly to opt-in to the new behaviour and silence this warning.

```
shap.summary_plot(sv_high, X_test_s, feature_names=feature_cols, show=False)
```



```
In [26]: mean_abs_shap = np.abs(sv_high).mean(axis=0)
importance_df = pd.DataFrame({'feature': feature_cols, 'mean_abs_shap': mean_abs_shap}).sort_values(
    'mean_abs_shap', ascending=False)
importance_df
```

Out[26]:

	feature	mean_abs_shap
3	BS	0.757268

Fig. 7 – Per-class test report and global SHAP summary plot (high-risk class)

4.2 Local Explainability – Synthetic Record

- Input: Age=39, SystolicBP=145, DiastolicBP=95, BS=12.8, BodyTemp=98.6, HeartRate=78.
- Model output: P(low)=0.001, P(mid)=0.013, P(high)=0.986 → predicted class = high risk.
- Local SHAP: SystolicBP is the single largest positive contributor for this record, followed by BS and Age; PulsePressure, HeartRate and BS_Age_Ratio push slightly the other way but are outweighed.
- This is the format meant for a health worker – the 2–3 readings driving the flag, not just a bare label.

1	SystolicBP	0.713572
4	BodyTemp	0.349592
0	Age	0.284443
9	BS_Age_Ratio	0.187310
6	PulsePressure	0.181529
5	HeartRate	0.152733
7	MAP	0.151611
2	DiastolicBP	0.066214
8	AgeBand	0.000000

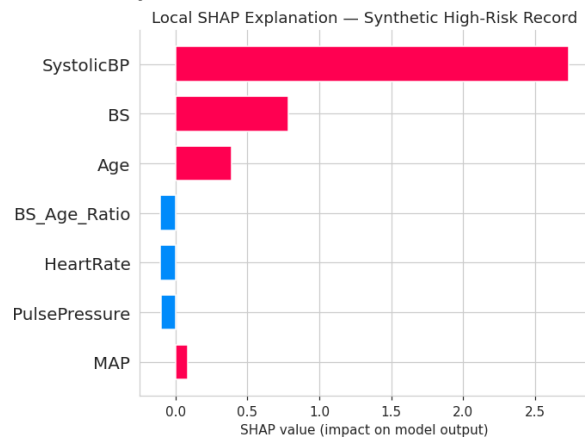
```
In [27]: # Local explanation on one synthetic high-risk record
synthetic = pd.DataFrame([
    'Age': 39, 'SystolicBP': 145, 'DiastolicBP': 95, 'BS': 12.8, 'BodyTemp': 98.6, 'HeartRate': 78,
])
synthetic['PulsePressure'] = synthetic['SystolicBP'] - synthetic['DiastolicBP']
synthetic['MAP'] = synthetic['DiastolicBP'] + synthetic['PulsePressure']/3
synthetic['AgeBand'] = synthetic['Age'].apply(age_band)
synthetic['BS_Age_Ratio'] = synthetic['BS'] / synthetic['Age']
synthetic = synthetic[feature_cols]
synthetic_s = pd.DataFrame(scaler.transform(synthetic), columns=feature_cols)

proba = best_xgb.predict_proba(synthetic_s)[0]
pred_class = ['low risk', 'mid risk', 'high risk'][np.argmax(proba)]
print("Predicted probabilities [low, mid, high]:", np.round(proba,3))
print("Predicted class:", pred_class)

sv_local = explainer.shap_values(synthetic_s)
sv_local_high = sv_local[2] if isinstance(sv_local, list) else sv_local[:, 2]

plt.figure()
shap.bar_plot(sv_local_high[0], feature_names=feature_cols, show=False)
plt.title("Local SHAP Explanation – Synthetic High-Risk Record")
plt.tight_layout()
plt.savefig("fig_shap_local.png", dpi=120, bbox_inches='tight')
plt.show()

Predicted probabilities [low, mid, high]: [0.001 0.013 0.986]
Predicted class: high risk
```



13. Ethics and Responsible-Use Notes

- **Bias/fairness:** dataset drawn from one country/period (rural Bangladesh, IoT clinics); no protected attributes present besides Age, which was checked and is always used in combination with BS/BP rather than alone.
- **Privacy:** no directly identifying fields in the source data; any deployment must still treat vitals as sensitive health data.
- **False negatives vs false positives:** a missed high-risk case is far costlier than a false alarm — class weighting

Fig. 8 – SHAP feature-importance table, local explanation on the synthetic record, and ethics notes

4.3 Bias and Fairness

- No protected attributes (ethnicity, income, etc.) exist in the data – the only demographic field is Age.

- Checked whether Age acts as a shortcut: Age is always used in combination with BS/BP in the SHAP interaction pattern, not as a stand-alone deciding factor – consistent with real clinical reasoning rather than a spurious proxy.
- Main fairness risk is representativeness, not within-data bias: single country, rural clinics, one collection window.

4.4 Privacy

- Source dataset has no directly identifying fields; 6 anonymised vitals are not realistically re-identifiable in isolation.
- Any real deployment must still treat records as sensitive health data under local regulation (secure storage, no linkage to identity without consent).

4.5 Uncertainty and Error Costs

- False negative (true high-risk → predicted low/mid) is far costlier than a false positive – addressed via `class_weight='balanced'` and `f1_macro`-based tuning rather than plain accuracy.
- Predicted probabilities (Section 4.2) are intended to be shown alongside the label, so a 55%-confidence flag is visibly different from a 98.6%-confidence one.

4.6 Human Oversight and Deployment Limits

- Triage/decision-support signal only – never an automated diagnosis or automated referral without a human in the loop.
- 451 cleaned records is small for a 3-class clinical model – re-validation required before use outside a similar rural-clinic population.
- Weak mid-risk performance (Section 3.5) means mid-risk predictions specifically should be treated with lower confidence by the health worker than low/high predictions.
- Recommend logging real-world prediction–outcome pairs post-deployment to audit high-risk recall periodically rather than trusting the offline test numbers indefinitely.